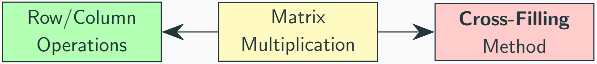
Cross-Filling Method

Finding Hidden Patterns in Recipe Tables

Introduction

Learning Objectives

Logic maps for today's cross-filling method's place in the big picture:



Key Questions:

- Can we **discover hidden intermediate products** from a direct recipe table?
- What is a **simple pattern** (rank-one table)?
- How to **decompose** complex tables into simple patterns?
- How to convert between **sum** and **product** forms?

Review: Sum-of-Rank-One View

Recall from Lecture 1, matrix multiplication $C = AB$ has three views:

$$C = \begin{pmatrix} | & | & & | \\ a_1 & a_2 & \cdots & a_n \\ | & | & & | \end{pmatrix} \begin{pmatrix} - & b_1^T & - \\ - & b_2^T & - \\ \vdots & \vdots & \vdots \\ - & b_n^T & - \end{pmatrix} = \sum_{k=1}^{n} a_k \, b_k^T$$

Each $a_k \, b_k^T$ is called a **rank-one piece** (column $\times$ row).

Today's question: Can we **reverse** this? Given C, find the pieces?

Discovering Simple Patterns (Concretization)

The Motivating Problem

Coffee shop scenario:

We have a **direct** recipe table from raw materials to final products:

$$C = \begin{array}{c|cc} & 🍱 & 🍲 \\ \hline 🍃 & 2 & 2 \\ 🍋 & 1 & 1 \\ 🫘 & 0 & 4 \\ 🐄 & 2 & 1 \end{array}$$

Question: Are there **hidden intermediate products** (like 🥛, ☕, 🍵) that we could use to simplify this?

What is a Simple Pattern?

Look at the first two rows of our table:

	🍱	🍲
🍃	2	2
🍋	1	1

Notice: Both products use **the same amount**!

- 🍱: 2🍃 + 1🍋
- 🍲: 2🍃 + 1🍋

Can we extract this **common base** and see what's different?

Setup: The Full Direct Table

$C =$

		
	2	2
	1	1
	0	4
	2	1

We noticed: first two rows have the same pattern.

Can we **extract** this pattern and see what's left?

Change: Choose a Pivot

Pick a pivot (center of our cross):

Let's pick the 🍃 row and 🍱 column. Entry $= 2$.

$C =$

	🍱	🍲
🍃	**2**	2
🍋	1	1
🫘	0	4
🐄	2	1

This pivot will be the **center of our cross**.

Calculation Step 1: Extract the Cross

The **cross** through the pivot includes:

- Entire 🍃 row (row through pivot)
- Entire 🍱 column (column through pivot)

$C =$

	🍱	🍲
🍃	**2**	2
🍋	1	1
🫘	0	4
🐄	2	1

Cross row: $(2, 2)$ Cross column: $(2, 1, 0, 2)$

Calculation Step 2: Build the Pattern Table

Idea: Use the cross to fill all entries via "multiplication table":

$$\text{New entry} = \frac{(\text{column value}) \times (\text{row value})}{\text{pivot}}$$

For 🍲 and 🍋:

$$\text{Entry} = \frac{\color{blue}{1} \times \color{blue}{2}}{\color{red}{\mathbf{2}}} = 1$$

For 🍲 and :

$$\text{Entry} = \frac{\color{blue}{0} \times \color{blue}{2}}{\color{red}{\mathbf{2}}} = 0$$

Result: The First Pattern R_1

$R_1 =$

	🍱	🍲
🍃	2	2
🍋	1	1
🫘	0	0
🐄	2	2

Interpretation: This is the **common base recipe**!

- Base recipe: 2🍃 + 1🍋 + 0🫘 + 2🐄
- 🍱 uses: 1 base
- 🍲 uses: 1 base (+ adjustments)

Insight: What is This Pattern?

The pattern R_1 is a **rank-one matrix**:

$$R_1 = \begin{pmatrix} 2 \\ 1 \\ 0 \\ 2 \end{pmatrix} \begin{pmatrix} 1 & 1 \end{pmatrix}$$

- **Column** $(2, 1, 0, 2)$: the base recipe - **Row** $(1, 1)$: both products use 1 this base

Key: **One pattern, repeated!** Both columns are the same.

Setup: What's Left After R_1?

Subtract R_1 from the original table C:

$C - R_1 =$

	🍱	🍲
🍃	$2-2$	$2-2$
🍋	$1-1$	$1-1$
🫘	$0-0$	$4-0$
🐄	$2-2$	$1-2$

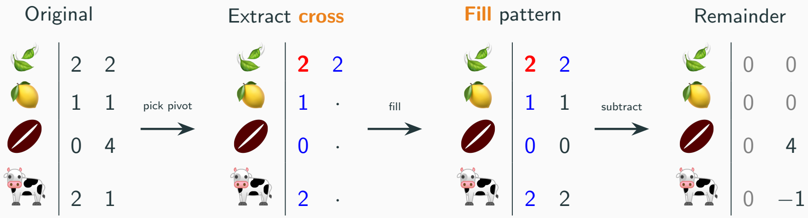
Result: The Remainder

$C_2 = C - R_1 =$

	🍱	🍲
🍃	0	0
🍋	0	0
🫘	0	4
🐄	0	-1

What's left? - 🍱 column: all zeros (base recipe exactly matches!) - 🍲 column: adjustments needed ($+4$ 🫘, -1 🐄)

Insight: Why "Cross-Filling"?



The method **fills in** the cross region, then eliminates it!

Change: Find Second Pattern

Look at the remainder:

$$C_2 = \begin{array}{c|cc} & \text{🍱} & \text{🍲} \\ \hline \text{🫘} & 0 & 4 \\ \text{🐄} & 0 & -1 \end{array}$$

This is the **adjustment** for 🍲: +4 🫘, -1 🐄

Pick pivot: 🫘 row, 🍲 column. Entry $= 4$.

Calculation: Fill Second Cross

Pivot at (3,2) $= 4$. Cross: column $(0, 0, 4, -1)^T$, row $(0, 4)$

$$R_2 = \frac{1}{\mathbf{4}} \begin{pmatrix} 0 \\ 0 \\ 4 \\ -1 \end{pmatrix} \begin{pmatrix} 0 & 4 \end{pmatrix} =$$

	🍱	🍲
🍃	0	0
🍋	0	0
	0	**4**
🐄	0	-1

Interpretation: The **adjustment term** only affects 🍲.

Result: Complete Decomposition

$C_2 - R_2 =$

	🍱	🍲
🫘	0	0
🐄	0	0

Done! Remainder is zero.

The Full Decomposition

We discovered:

$$C = R_1 + R_2$$

$$\begin{array}{c|cc} \text{🌿} & 2 & 2 \\ \text{🍋} & 1 & 1 \\ \text{🫘} & 0 & 4 \\ \text{🐄} & 2 & 1 \end{array} = \begin{array}{c|cc} \text{🌿} & 2 & 2 \\ \text{🍋} & 1 & 1 \\ \text{🫘} & 0 & 0 \\ \text{🐄} & 2 & 2 \end{array} + \begin{array}{c|cc} \text{🌿} & 0 & 0 \\ \text{🍋} & 0 & 0 \\ \text{🫘} & 0 & 4 \\ \text{🐄} & 0 & -1 \end{array}$$

Two **rank-one patterns**: base recipe + adjustment!

Insight: Base + Adjustment Decomposition

What we found:

$$🍲 = 1 \times (\text{base recipe}) + \text{adjustment}$$

Key insight:

- R_1: Common base shared by both products
- R_2: Adjustment term (only for 🍲)
- 🍱 = exactly the base recipe
- 🍲 = base + extra 4 🫘 - 1 🐄

This is **expressing complex recipes in terms of simpler patterns**!

Change of Perspective

Change of Perspective

What we've learned so far:

- **What** a simple pattern (rank-one) looks like one base mix used in different amounts
- **How** to extract patterns using cross-filling pick pivot, fill cross, subtract
- **Why** this works discovers hidden intermediate products

From This Point Forward

We **forget about coffee shops** and work with **abstract matrices**:

- **What**: Matrices A, B, C, rank-one pieces R_k
- **How**: Cross-filling algorithm, sum $\leftrightarrow$ product conversion
- **Why**: Compute rank, factorizations, solve equations

This simplifies notation while preserving the intuition.

Rank-One Matrices (Formalization)

Definition: Rank-One Matrix

Definition (Rank-One Matrix)

A matrix R is **rank-one** if it can be written as

$$R = \mathbf{u}\,\mathbf{v}^T$$

for some nonzero column vector $\mathbf{u}$ and nonzero row vector $\mathbf{v}^T$.

Example:

$$R = \begin{pmatrix} 2 \\ 1 \\ 3 \end{pmatrix} \begin{pmatrix} 1 & 2 \end{pmatrix} = \begin{pmatrix} 2 & 4 \\ 1 & 2 \\ 3 & 6 \end{pmatrix}$$

Key: All columns are multiples of $\mathbf{u}$. All rows are multiples of $\mathbf{v}^T$.

Recognizing Rank-One Structure

Is this matrix rank-one?

$$\begin{pmatrix} 6 & 9 & 3 \\ 4 & 6 & 2 \\ 10 & 15 & 5 \end{pmatrix}$$

Check rows:

- Row 1: $(6, 9, 3) = 3 \cdot (2, 3, 1)$
- Row 2: $(4, 6, 2) = 2 \cdot (2, 3, 1)$
- Row 3: $(10, 15, 5) = 5 \cdot (2, 3, 1)$

Yes! $R = \begin{pmatrix} 3 \\ 2 \\ 5 \end{pmatrix} \begin{pmatrix} 2 & 3 & 1 \end{pmatrix}$

Cross-Product Property

In a rank-one matrix $R = \mathbf{u}\,\mathbf{v}^T$, every entry is $r_{ij} = u_i v_j$.

Visual meaning of subscripts:

$$
\begin{array}{rc}
 & \begin{array}{ccc} \phantom{r_{11}} & \text{col } j & \phantom{r_{13}} \\ & \uparrow & \end{array} \\
\leftarrow \text{row } i & \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & \boxed{r_{22}} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}
\end{array}
$$

$r_{ij} = \text{row } i\text{, column } j$

Cross-Product Property

For any four entries forming a rectangle:

$$\boxed{r_{ij} \cdot r_{k\ell} = r_{i\ell} \cdot r_{kj}}$$

$$\begin{pmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & r_{ij} & \cdot & r_{i\ell} \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & r_{kj} & \cdot & r_{k\ell} \end{pmatrix}$$

$$r_{ij} \cdot r_{k\ell} = r_{i\ell} \cdot r_{kj}$$

Why? $(u_i v_j)(u_k v_\ell) = (u_i v_\ell)(u_k v_j) = u_i u_k v_j v_\ell$

Not Every Matrix is Rank-One

$$C = \begin{pmatrix} 2 & 4 \\ 1 & 2 \\ 3 & 7 \end{pmatrix}$$

Check ratios: $\frac{4}{2} = 2, \quad \frac{2}{1} = 2, \quad \frac{7}{3} \neq 2 \quad \leftarrow$ Different!

This is **not rank-one**. But can we write it as a **sum** of rank-one pieces?

$$C = \underbrace{\begin{pmatrix} ? \end{pmatrix}}_{R_1} + \underbrace{\begin{pmatrix} ? \end{pmatrix}}_{R_2} + \cdots$$

Cross-filling answers this!

Cross-Filling Algorithm

The Cross-Filling Algorithm

Proposition (Cross-Filling Algorithm)

Input: Matrix A ($m \times n$)

Output: $A = R_1 + R_2 + \cdots + R_r$ (sum of rank-one matrices)

Procedure:

1. Find any nonzero entry $a_{ij} \neq 0$ (the **pivot**)

$$\begin{array}{cc} & \begin{array}{cccc} \phantom{a_{11}} & \text{col } j & & \phantom{a_{1n}} \end{array} \\ \begin{array}{c} \\ \\ \\ \\ \text{row } i \\ \\ \\ \\ \\ \end{array} & \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{i1} & \boxed{a_{ij}} & \cdots & a_{in} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} \end{array}$$

row i, col j

Cross-Filling Algorithm (continued)

Proposition (Cross-Filling Algorithm (continued))

1. Extract the cross: row i and column j
2. Form rank-one piece:

$$R = \frac{1}{a_{ij}} \cdot (\text{column } j) \cdot (\text{row } i)$$

3. Compute remainder: $A' = A - R$
4. Repeat on A' until $A' = O$

Each step eliminates one cross (one row and one column become zero).

Pivot Selection Principles

Key Question: Which nonzero entry should we pick as pivot?

Pivot Independence Principle

At each step, choose any nonzero entry $a_{ij} \neq 0$ such that:

- Row i is **not yet zeroed** by previous steps
- Column j is **not yet zeroed** by previous steps

Consequence: **Many different pivot sequences are valid!**

- Diagonal pivots $(1,1) \to (2,2) \to \cdots$ one choice
- Off-diagonal pivots $(1,3) \to (2,1) \to \cdots$ also valid!
- Different sequences give different decompositions, but **same rank**

Example: Diagonal Cross-Filling

Decompose using diagonal pivots:

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 3 & 3 \\ 1 & 3 & 8 \end{pmatrix}$$

Strategy: Pick pivots $(1,1) \to (2,2) \to (3,3)$

We will see this produces **LU decomposition** structure.

Iteration 1: Pivot $a_{11} = 1$

Choosing position $(1, 1)$:

$$
\begin{array}{cc}
 & \text{col 1} \\
\text{row 1} & \begin{pmatrix} \boxed{a_{11}} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}
\end{array}
$$

Highlight the cross:

$$
A = \begin{pmatrix} \mathbf{1} & 1 & 1 \\ 1 & 3 & 3 \\ 1 & 3 & 8 \end{pmatrix}
$$

Iteration 1: Fill the Cross

Fill using the cross:

$$R_1 = \frac{1}{\mathbf{1}} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{1} & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Iteration 1: Remainder A_1

$$A_1 = A - R_1 = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 3 & 3 \\ 1 & 3 & 8 \end{pmatrix} - \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 2 & 2 \\ 0 & 2 & 7 \end{pmatrix}$$

Row 1 and column 1 are now zero. ✓

Iteration 2: Pivot $(A_1)_{22} = 2$

Choosing position $(2, 2)$ in the remainder:

$$\begin{array}{cc} & \begin{array}{ccc} & \text{col } 2 & \end{array} \\ \text{row } 2 & \begin{pmatrix} 0 & 0 & 0 \\ 0 & \boxed{a_{22}} & a_{23} \\ 0 & a_{32} & a_{33} \end{pmatrix} \end{array}$$

$$A_1 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & \mathbf{2} & 2 \\ 0 & 2 & 7 \end{pmatrix}$$

Iteration 2: Fill the Cross

Fill:

$$R_2 = \frac{1}{\mathbf{2}} \begin{pmatrix} 0 \\ 2 \\ 2 \end{pmatrix} \begin{pmatrix} 0 & 2 & 2 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & \mathbf{2} & 2 \\ 0 & 2 & 2 \end{pmatrix}$$

Iteration 2: Remainder A_2

$$A_2 = A_1 - R_2 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 2 & 2 \\ 0 & 2 & 7 \end{pmatrix} - \begin{pmatrix} 0 & 0 & 0 \\ 0 & 2 & 2 \\ 0 & 2 & 2 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 5 \end{pmatrix}$$

Rows 1–2 and columns 1–2 are now zero. ✓

Iteration 3: Final Piece

$$A_2 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \mathbf{5} \end{pmatrix}$$

This is already rank-one:

$$R_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & 0 & 5 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \mathbf{5} \end{pmatrix}$$

Remainder: $A_3 = A_2 - R_3 = O$. **Done!**

Full Decomposition (Diagonal Pivots)

$$A = \underbrace{\begin{pmatrix} \mathbf{1} & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}}_{R_1} + \underbrace{\begin{pmatrix} 0 & 0 & 0 \\ 0 & \mathbf{2} & 2 \\ 0 & 2 & 2 \end{pmatrix}}_{R_2} + \underbrace{\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \mathbf{5} \end{pmatrix}}_{R_3}$$

Three rank-one pieces:

- R_1: pivot **1** at position $(1, 1)$
- R_2: pivot **2** at position $(2, 2)$
- R_3: pivot **5** at position $(3, 3)$

Key: **Diagonal** pivot strategy $\rightarrow$ special structure (LU decomposition)

Example: Non-Diagonal Cross-Filling (Setup)

Now let's decompose the **same matrix** using **off-diagonal pivots**:

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 3 & 3 \\ 1 & 3 & 8 \end{pmatrix}$$

New strategy: Pick pivots $(1, 3) \to (2, 1) \to (3, 2)$ (all off-diagonal!)

Question: Will we get a different decomposition?

Answer: Yes! Different R_k pieces, but **same rank** (still 3 pieces).

Non-Diagonal Iteration 1: Pivot $(1, 3)$

Choose position $(1, 3)$: entry $= 1$

$$A = \begin{pmatrix} 1 & 1 & \mathbf{1} \\ 1 & 3 & 3 \\ 1 & 3 & 8 \end{pmatrix} \quad \text{Cross: row } 1 = (1, 1, \mathbf{1}), \text{ col } 3 = (\mathbf{1}, 3, 8)^T$$

Fill:

$$R_1' = \frac{1}{\mathbf{1}} \begin{pmatrix} \mathbf{1} \\ 3 \\ 8 \end{pmatrix} \begin{pmatrix} 1 & 1 & \mathbf{1} \end{pmatrix} = \begin{pmatrix} \mathbf{1} & 1 & 1 \\ 3 & 3 & 3 \\ 8 & 8 & 8 \end{pmatrix}$$

Remainder:

$$A_1' = A - R_1' = \begin{pmatrix} 0 & 0 & 0 \\ -2 & 0 & 0 \\ -7 & -5 & 0 \end{pmatrix}$$

Row 1 and column 3 are zero. ✓

Non-Diagonal Iteration 2: Pivot $(2,1)$

Choose position $(2,1)$ in A'_1: entry $= -2$

$$A'_1 = \begin{pmatrix} 0 & 0 & 0 \\ \mathbf{-2} & 0 & 0 \\ -7 & -5 & 0 \end{pmatrix}$$

Fill:

$$R'_2 = \frac{1}{\mathbf{-2}} \begin{pmatrix} 0 \\ \mathbf{-2} \\ -7 \end{pmatrix} \begin{pmatrix} \mathbf{-2} & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ \mathbf{-2} & 0 & 0 \\ -7 & 0 & 0 \end{pmatrix}$$

Remainder:

$$A'_2 = A'_1 - R'_2 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & -5 & 0 \end{pmatrix}$$

Non-Diagonal Iteration 3: Final Piece

$$A_2' = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & \mathbf{-5} & 0 \end{pmatrix}$$

$$R_3' = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & -5 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & \mathbf{-5} & 0 \end{pmatrix}$$

Done! $A = R_1' + R_2' + R_3'$

Comparison: Diagonal vs Non-Diagonal

Same matrix A, different pivot strategies:

	Diagonal $(1,1) \to (2,2) \to (3,3)$	**Off-diagonal** $(1,3) \to (2,1) \to (3$
R_1	$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 1 & 1 \\ 3 & 3 & 3 \\ 8 & 8 & 8 \end{pmatrix}$
R_2	$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 2 & 2 \\ 0 & 2 & 2 \end{pmatrix}$	$\begin{pmatrix} 0 & 0 & 0 \\ -2 & 0 & 0 \\ -7 & 0 & 0 \end{pmatrix}$
R_3	$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 5 \end{pmatrix}$	$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & -5 & 0 \end{pmatrix}$
Rank	**3**	**3**

Key: **Different decompositions, same rank!**

Another Example: 3×4 Matrix

Decompose with non-diagonal pivots:

$$B = \begin{pmatrix} 2 & 0 & 4 & 2 \\ 1 & 3 & 5 & 4 \\ 0 & 6 & 2 & 5 \end{pmatrix}$$

Strategy: $(2,3) \to (1,1) \to (3,2)$

Step 1: Pivot at $(2,3)$, entry $= 5$

Cross: row $2 = (1,3,5,4)$, col $3 = (4,5,2)^T$

$$R_1 = \frac{1}{5} \begin{pmatrix} 4 \\ 5 \\ 2 \end{pmatrix} \begin{pmatrix} 1 & 3 & 5 & 4 \end{pmatrix} = \begin{pmatrix} 4/5 & 12/5 & 4 & 16/5 \\ 1 & 3 & 5 & 4 \\ 2/5 & 6/5 & 2 & 8/5 \end{pmatrix}$$

3×4 **Remainder After Step 1**

$$B_1 = B - R_1 = \begin{pmatrix} 6/5 & -12/5 & 0 & -6/5 \\ 0 & 0 & 0 & 0 \\ -2/5 & 24/5 & 0 & 17/5 \end{pmatrix}$$

Row 2 and column 3 are zero.

Step 2: Pivot at $(1, 1)$, entry $= 6/5$

$$R_2 = \frac{5}{6} \begin{pmatrix} 6/5 \\ 0 \\ -2/5 \end{pmatrix} \begin{pmatrix} 6/5 & -12/5 & 0 & -6/5 \end{pmatrix} = \begin{pmatrix} 6/5 & -12/5 & 0 & -6/5 \\ 0 & 0 & 0 & 0 \\ -2/5 & 4/5 & 0 & 2/5 \end{pmatrix}$$

$$B_2 = B_1 - R_2 = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 3 \end{pmatrix}$$

3×4 Final Step

$$B_2 = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 3 \end{pmatrix}$$

Step 3: Pivot at $(3, 2)$, entry $= 4$

$$R_3 = \frac{1}{4} \begin{pmatrix} 0 \\ 0 \\ 4 \end{pmatrix} \begin{pmatrix} 0 & 4 & 0 & 3 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 4 & 0 & 3 \end{pmatrix}$$

$B_3 = B_2 - R_3 = O$. **Done!**

Result: $B = R_1 + R_2 + R_3, \quad \mathrm{rank}(B) = 3$

Key Insight: Pivot Flexibility

Main Takeaway

Pivot selection is flexible!

- At each step, choose **any nonzero entry** in a non-zeroed row and column
- Different pivot choices $\rightarrow$ different rank-one pieces R_k
- But the **total number of pieces** ($=$ rank) is always the same

Why does this matter?

- **Computational**: Choose pivots to avoid small numbers (numerical stability)
- **Structural**: Diagonal pivots $\rightarrow$ LU decomposition
- **Theoretical**: Rank is well-defined (independent of decomposition method)

Next lecture: Prove that rank is well-defined via linear independence.

Definition: Rank of a Matrix

Definition (Rank)

The **rank** of a matrix A is the number of rank-one pieces in its cross-filling decomposition:

$$A = R_1 + R_2 + \cdots + R_r \quad \Longrightarrow \quad \text{rank}(A) = r$$

Examples:

- Zero matrix O: rank 0
- Rank-one matrix $\mathbf{uv}^T$: rank 1
- Identity I_n: rank n
- Our A above: rank 3 (both decompositions gave 3 pieces)
- The 3×4 matrix B: rank 3

Important: Different pivot choices give different decompositions, but always the same count r. (Proved in Lecture 4.)

Sum $\leftrightarrow$ Product Equivalence

From Sum to Product Form

Each rank-one piece is an **outer product**:

$$R_k = \mathbf{u}_k \, \mathbf{v}_k^T$$

From our diagonal example:

$$R_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 \end{pmatrix}$$

$$R_2 = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & 2 & 2 \end{pmatrix}$$

$$R_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & 0 & 5 \end{pmatrix}$$

Collecting the Pieces

Collect columns $\mathbf{u}_k$ into U, rows $\mathbf{v}_k^T$ into V:

$$U = \begin{pmatrix} | & | & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \mathbf{u}_3 \\ | & | & | \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

$$V = \begin{pmatrix} - & \mathbf{v}_1^T & - \\ - & \mathbf{v}_2^T & - \\ - & \mathbf{v}_3^T & - \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 2 & 2 \\ 0 & 0 & 5 \end{pmatrix}$$

By sum-of-rank-one: $UV = \sum_{k=1}^{3} \mathbf{u}_k \, \mathbf{v}_k^T = R_1 + R_2 + R_3 = A$

Verification

$$UV = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 \\ 0 & 2 & 2 \\ 0 & 0 & 5 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 3 & 3 \\ 1 & 3 & 8 \end{pmatrix} = A \quad \checkmark$$

Notice: U is **lower triangular**, V is **upper triangular**!

This happened because we chose **diagonal pivots** $(1, 1) \to (2, 2) \to (3, 3)$.

Key: Diagonal pivots $\implies$ **LU decomposition** (preview of next section).

Sum $\leftrightarrow$ Product Equivalence

Proposition (Sum $\leftrightarrow$ Product)

For any matrix A:

$$A = \sum_{k=1}^{r} \mathbf{u}_k \, \mathbf{v}_k^T \quad \Longleftrightarrow \quad A = U \cdot V$$

where U has columns $\mathbf{u}_k$ and V has rows $\mathbf{v}_k^T$.

Two directions:

- **Lecture 1**: Given product $A = UV$, read off sum $A = \sum \mathbf{u}_k \, \mathbf{v}_k^T$
- **Today**: Given sum from cross-filling, assemble product $A = UV$

The Inner Dimension

When $A = UV$ with A being $m \times n$:

$$A_{m \times n} = U_{m \times \boxed{r}} \cdot V_{\boxed{r} \times n}$$

The shared dimension r is the **inner dimension**.

- Outer dimensions m, n: visible (size of A)
- Inner dimension r: **hidden** = number of independent patterns

Key: $\text{rank}(A) = r =$ inner dimension of the factorization.

LU Decomposition via Diagonal Cross-Filling

What is LU Decomposition?

Goal: Write $A = LU$ where:

- L is **lower triangular** with **diagonal = 1**
- U is **upper triangular**

Example:

$$\begin{pmatrix} 2 & 4 & 6 \\ 1 & 5 & 7 \\ 3 & 9 & 10 \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{pmatrix}}_{L} \underbrace{\begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix}}_{U}$$

Key constraint: L must have 1's on the diagonal!

Method: Use **diagonal cross-filling** with a special technique.

The Key Technique: Normalizing L's Diagonal

Recall the cross-filling formula at pivot (k, k):

$$R_k = \frac{1}{a_{kk}} \cdot (\text{column } k) \cdot (\text{row } k)$$

Rewrite to separate the normalization:

$$R_k = \underbrace{\left(\frac{\text{column } k}{a_{kk}}\right)}_{\text{normalized column}} \cdot (\text{row } k)$$

Key observation:

- The k-th entry of the normalized column $= \dfrac{a_{kk}}{a_{kk}} = 1$
- Collect these normalized columns $\rightarrow L$ has diagonal $= 1$!
- Collect the original rows $\rightarrow U$ (upper triangular)

This is the **trick** that ensures L has diagonal $= 1$.

Example: 4×4 Matrix LU Decomposition

Decompose via diagonal cross-filling:

$$A = \begin{pmatrix} 2 & 4 & -2 & 2 \\ 4 & 9 & -3 & 5 \\ -2 & -3 & 7 & -1 \\ 2 & 5 & -1 & 6 \end{pmatrix}$$

Strategy: Diagonal pivots $(1,1) \to (2,2) \to (3,3) \to (4,4)$

We will extract R_1, R_2, R_3, R_4 and assemble L and U.

Step 1: Pivot at $(1,1)$, entry $= 2$

$$A = \begin{pmatrix} 2 & 4 & -2 & 2 \\ 4 & 9 & -3 & 5 \\ -2 & -3 & 7 & -1 \\ 2 & 5 & -1 & 6 \end{pmatrix}$$

Cross: column $= (2, 4, -2, 2)^T$, row $= (2, 4, -2, 2)$

Apply the technique:

$$R_1 = \underbrace{\frac{1}{2}\begin{pmatrix} 2 \\ 4 \\ -2 \\ 2 \end{pmatrix}}_{= \text{column 1 of } L} \cdot \underbrace{\begin{pmatrix} 2 & 4 & -2 & 2 \end{pmatrix}}_{= \text{row 1 of } U} = \underbrace{\begin{pmatrix} 1 \\ 2 \\ -1 \\ 1 \end{pmatrix}}_{\ell_1} \begin{pmatrix} 2 & 4 & -2 & 2 \end{pmatrix}$$

Note: ℓ_1 has first entry $= 1$ ✓

Remainder After Step 1

$$A_1 = A - R_1 = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 5 & 1 \\ 0 & 1 & 1 & 4 \end{pmatrix}$$

Row 1 and column 1 are zero.

Next: Pivot at $(2, 2)$ in the non-zero block.

Step 2: Pivot at $(2, 2)$, entry $= 1$

$$A_1 = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & \mathbf{1} & 1 & 1 \\ 0 & 1 & 5 & 1 \\ 0 & 1 & 1 & 4 \end{pmatrix}$$

Cross: column $= (0, 1, 1, 1)^T$, row $= (0, 1, 1, 1)$

Apply the technique:

$$R_2 = \underbrace{\frac{1}{1}\begin{pmatrix} 0 \\ 1 \\ 1 \\ 1 \end{pmatrix}}_{= \text{column 2 of } L} \cdot \underbrace{\begin{pmatrix} 0 & 1 & 1 & 1 \end{pmatrix}}_{= \text{row 2 of } U} = \underbrace{\begin{pmatrix} 0 \\ 1 \\ 1 \\ 1 \end{pmatrix}}_{\ell_2} \begin{pmatrix} 0 & 1 & 1 & 1 \end{pmatrix}$$

Note: ℓ_2 has second entry $= 1$ ✓

Remainder After Step 2

$$A_2 = A_1 - R_2 = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix}$$

Rows 1–2 and columns 1–2 are zero.

Next: Pivot at $(3, 3)$.

Step 3: Pivot at $(3,3)$, entry $= 4$

$$A_2 = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \mathbf{4} & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix}$$

Cross: column $= (0,0,4,0)^T$, row $= (0,0,4,0)$

Apply the technique:

$$R_3 = \underbrace{\frac{1}{4}\begin{pmatrix} 0 \\ 0 \\ 4 \\ 0 \end{pmatrix}}_{=\text{ column 3 of } L} \cdot \underbrace{\begin{pmatrix} 0 & 0 & 4 & 0 \end{pmatrix}}_{=\text{ row 3 of } U} = \underbrace{\begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}}_{\ell_3} \begin{pmatrix} 0 & 0 & 4 & 0 \end{pmatrix}$$

Note: ℓ_3 has third entry $= 1$ ✓

Step 4: Final Pivot at $(4, 4)$, entry $= 3$

$$A_3 = A_2 - R_3 = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \mathbf{3} \end{pmatrix}$$

$$R_4 = \underbrace{\frac{1}{3}\begin{pmatrix} 0 \\ 0 \\ 0 \\ 3 \end{pmatrix}}_{=\text{ column 4 of } L} \cdot \underbrace{\begin{pmatrix} 0 & 0 & 0 & 3 \end{pmatrix}}_{=\text{ row 4 of } U} = \underbrace{\begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}}_{\ell_4} \begin{pmatrix} 0 & 0 & 0 & 3 \end{pmatrix}$$

Note: ℓ_4 has fourth entry $= 1$ ✓

$A_4 = A_3 - R_4 = O.$ **Done!**

Assembling L and U

Collect normalized columns into L:

$$L = \begin{pmatrix} | & | & | & | \\ \ell_1 & \ell_2 & \ell_3 & \ell_4 \\ | & | & | & | \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ -1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{pmatrix}$$

Key: Diagonal $= (1, 1, 1, 1)$ ✓

Collect rows into U:

$$U = \begin{pmatrix} - & \text{row 1} & - \\ - & \text{row 2} & - \\ - & \text{row 3} & - \\ - & \text{row 4} & - \end{pmatrix} = \begin{pmatrix} 2 & 4 & -2 & 2 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix}$$

Key: Upper triangular, pivots on diagonal ✓

Verification: $LU = A$

$$LU = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ -1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & 4 & -2 & 2 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix} ?= \begin{pmatrix} 2 & 4 & -2 & 2 \\ 4 & 9 & -3 & 5 \\ -2 & -3 & 7 & -1 \\ 2 & 5 & -1 & 6 \end{pmatrix}$$

First row: $(1)(2,4,-2,2) = (2,4,-2,2)$ ✓

Second row:
$(2)(2,4,-2,2) + (1)(0,1,1,1) = (4,8,-4,4) + (0,1,1,1) = (4,9,-3,5)$
✓

Third row: $(-1)(2,4,-2,2) + (1)(0,1,1,1) + (1)(0,0,4,0) =$
$(-2,-4,2,-2) + (0,1,1,1) + (0,0,4,0) = (-2,-3,7,-1)$ ✓

Fourth row:
$(1)(2,4,-2,2) + (1)(0,1,1,1) + (0)(0,0,4,0) + (1)(0,0,0,3) =$
$(2,5,-1,6)$ ✓

Why Does This Work?

The magic: By normalizing each column by its pivot, we ensure:

$$\ell_k = \frac{\text{column } k}{a_{kk}} \quad \Longrightarrow \quad (\ell_k)_k = \frac{a_{kk}}{a_{kk}} = 1$$

In general:

- Cross-filling with diagonal pivots: $A = R_1 + R_2 + \cdots + R_n$
- Each $R_k = \ell_k \cdot (\text{row } k)$ where $\ell_k = (\text{col } k)/a_{kk}$
- Collect: $L = [\ell_1 \mid \ell_2 \mid \cdots \mid \ell_n]$, rows into U
- Result: $A = LU$ with L lower triangular (diagonal $= 1$), U upper triangular

Summary:

$$\boxed{\text{Diagonal cross-filling} + \text{normalization trick} = \text{LU decomposition}}$$

When Diagonal LU Fails

Diagonal cross-filling requires **nonzero diagonal pivots**. But this can fail:

$$A = \begin{pmatrix} 0 & 3 \\ 1 & 2 \end{pmatrix}$$

Problem: position $(1, 1)$

$$\begin{pmatrix} \boxed{a_{11}} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$$

Pivot $a_{11} = 0$ cannot start!

Even later pivots can fail:

$$\begin{pmatrix} 1 & 2 & 3 \\ 1 & 2 & 4 \\ 1 & 3 & 5 \end{pmatrix} \xrightarrow{\text{after } R_1} \begin{pmatrix} 0 & 0 & 0 \\ 0 & \mathbf{0} & 1 \\ 0 & 1 & 2 \end{pmatrix}$$

Summary

Summary

Today we learned:

1. **Rank-one matrix**: $R = \mathbf{u}\,\mathbf{v}^T$ (one pattern, repeated)
2. **Cross-filling algorithm**: Decompose any matrix into rank-one pieces

$$A = R_1 + R_2 + \cdots + R_r$$

   Pick pivot → Extract cross → Fill → Subtract → Repeat
3. **Pivot selection principle**: Choose any nonzero entry in the remaindering matrix → flexible decompositions
4. **Pivot flexibility**: Different pivot sequences → different decompositions, same rank
5. **Sum ↔ Product equivalence**:

$$\sum_{k=1}^{r} \mathbf{u}_k\,\mathbf{v}_k^T = A = UV$$

6. **Rank** = number of pieces = inner dimension of UV
7. **Special case**: Diagonal pivots → LU decomposition

Looking Ahead

Lecture 4 (next week):

- **Well-definedness of rank** Why does rank not depend on pivot choices?
- **Linear independence** and **basis** Two languages for subspaces
- **Dimension** Connection to rank via trace
- Solving $Ax = b$ using cross-filling

The big picture:

Cross-filling $\rightarrow$ Rank $\rightarrow$ Subspaces $\rightarrow$ Projections $\rightarrow$ Spectral Decomposition

Every major topic connects back to rank-one decomposition!